Virtual Machine in C

Mykyta 'Nick' Khomiakov

March 25, 2026

Contents

1 Abstract **2**

2 Project Setup **3**

2.1 Prerequisites 3

2.2 Build 3

2.3 File Directory 4

2.4 Separation of Concerns 4

1 Abstract

This project outlines a stack-based virtual machine. Built from ground-up from C (C11 standard). The machine allows for 25 opcodes including operations such as: arithmetic, logic, control flow, functions, stack manipulations, call stack and register operations; the project also contains a built-in development shell (REPL) in terminal with four built-in features: code compilation, comments, opcode reference and an array index list. The REPL features don't affect operations performed in the virtual machine, rather documents those.

The primary motivation for developing this project was curiosity-driven investigation of both virtual machine design and under-the-hood operations of CPUs and programming languages.

Most CPUs one interacts with every day are register-based: they shuffle values between a fixed set of named registers (like `eax`, `rbx` etc.) before operating on them. A stack-based VM takes a different approach: operands are implicitly the top values on a stack, and instructions consume and produce values there.

This design was chosen deliberately for several reasons:

- **Simplicity** of implementation: There is no register-allocation problem to solve. Every opcode knows exactly where its inputs are (the top of the stack) and where to leave its output (back on the stack). The interpreter loop is a clean switch over a flat integer array, hence no complex decode stage needed.

- **Simplicity** of the instruction encoding: Since operands are implicit, most instructions are a single word (one integer). Only instructions that need a literal value – `PUSH`, `JMP`, `LD`, `ST`, `CALL` carry an extra word. This keeps bytecode compact and the assembler trivial to write.

- **Portability**: The entire VM state is a single artificial data structure holding plain integer arrays. There are no platform-specific register intrinsics, no inline assembly, and no OS dependencies. The same code compiles and runs identically on Linux, macOS, and Windows due to application of CMake.

- **Pedagogical clarity**: Stack machines map directly onto how many high-level language expressions are naturally evaluated (postfix / reverse-Polish notation). The execution model is easy to reason about step-by-step, making this a good platform for learning how interpreters and compilers work.

The trade-off is that stack machines can require more instructions than a register machine to express the same computation (e.g., `DUP` and `SWAP` are needed to reorder operands). This follows a similar idea of RISC CPU architecture, where the instruciton set is reduced for notably smaller and more energy-efficient CPUs. For this project, which is a teaching and experimentation, that is an acceptable cost.

2 Project Setup

2.1 Prerequisites

The project used CMake 4.1 and C11 standards for CLang in its setup. However, in order to run the project on any computer, it should have installed dependencies such as: GCC/CLang with C11 support. GNUMake is required for CMake build on Linux, however CMake can be comfortbaly ran from both Mac and Windows. This document recommends using environments such as Visual Studio for Windows, and CLion for Mac and Linux.

2.2 Build

In order to build the project from terminal, this command can be pasted into the terminal with the set directory of the project:

```
gcc -std=c11 -Wall -Wextra -Iinclude \
    src/main.c src/vm.c \
    src/io/lexer.c src/io/assembler.c \
    src/operations/maths.c src/operations/stack.c \
    src/operations/flow.c src/operations/logic.c \
    src/operations/registers.c \
    -o vm
```

In order to run the project either the IDE can handle the runtime, or the project can be ran via the following command:

```
./vm
```

2.3 File Directory

```
vm/
├── docs/
│   ├── sections/
│   │   ├── abstract.tex
│   │   └── project_setup.tex
│   │
│   ├── main.tex
│   └── main.pdf
│
├── src/
│   ├── main.c                # Interactive REPL entry point
│   ├── vm.c                  # Fetch-decode-execute loop; push/pop primitives
│   ├── io/
│   │   ├── lexer.c           # Tokenises a line of source text
│   │   └── assembler.c       # Translates tokens into bytecode words
│   └── operations/
│       ├── flow.c            # JMP, JZ, JNZ, CALL, RET
│       ├── logic.c           # EQ, LT, GT, AND, OR, NOT
│       ├── maths.c           # ADD, SUB, MULT, DIV, MOD
│       ├── registers.c       # LD, ST
│       └── stack.c           # PUSH, POP, DUP, SWAP, PEEK, PRINT, HALT
│
├── include/
│   ├── vm.h                  # VM struct, opcode enum, push/pop declarations
│   ├── io/
│   │   ├── lexer.h
│   │   └── assembler.h
│   └── operations/
│       ├── flow.h
│       ├── logic.h
│       ├── maths.h
│       ├── registers.h
│       └── stack.h
│
├── README.md
└── .gitignore
```

Figure 1: File Directory of the project

2.4 Separation of Concerns

Each operation category lives in its own translation unit. Adding a new opcode means touching only the relevant file plus a single case in vm.c – the rest of the codebase is unaffected. This was achieved via a divide and conquer approach, where modules that don't have to know about each other – don't know about each other. This separation of concerns allows for a simplified scalable design.